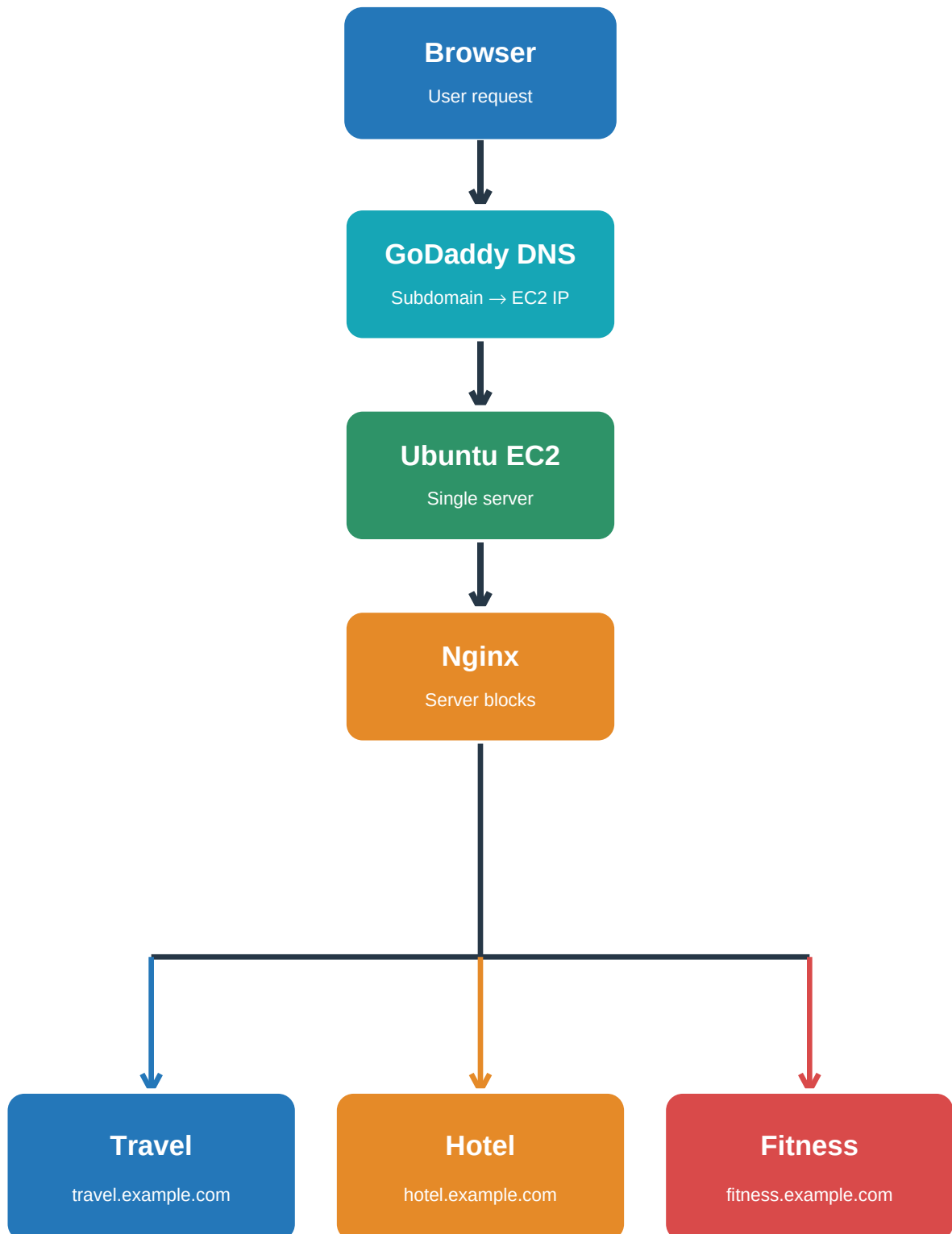


Ubuntu Nginx Multi-Site Deployment

One Ubuntu EC2 Server • GoDaddy DNS • Nginx • Three Static Websites

Deployment Architecture

Subdomain request → DNS resolution → EC2 → Nginx server block → matching website



Nginx routes each hostname to its matching website root

Travel • Hotel • Fitness are hosted independently on the same Ubuntu EC2 server.

Project Description

Ubuntu Nginx Multi-Site Deployment

This project demonstrates how three independent static websites can be hosted on a single Ubuntu EC2 server using Nginx. GoDaddy DNS is used for the subdomains, while Nginx server blocks route each hostname to the correct website directory. The websites are Travel, Hotel and Fitness, and the deployment also includes basic shell scripting and website verification.

Project Objectives

- Host three static websites on one Ubuntu EC2 instance.
- Configure Nginx server blocks for separate websites.
- Use GoDaddy DNS to connect subdomains to the EC2 public IP.

Websites Created

- Travel website
- Hotel website
- Fitness website

Technologies Used

- AWS EC2
- Ubuntu Linux
- Nginx
- GoDaddy DNS
- Shell Scripting
- HTML
- Git / GitHub

Important Paths

- /var/www/html/travel/
- /var/www/html/hotel/
- /var/www/html/fitness/
- /etc/nginx/sites-enabled/

Project Result

- All three websites are kept in separate directories.
- Each subdomain is handled by its own Nginx server block.
- One Ubuntu EC2 server is used to host the complete setup.

Step 1 — Launch the Ubuntu EC2 Instance

<https://us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:>

aws [Alt+S]

United States (N. Virginia) Prem Dhumal (026777907119) Prem Dhumal

EC2 > Instances > Launch an instance

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags [Info](#)

Name

Ubuntu Nginx Multisite [Add additional tags](#)

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Recents

Quick Start

Amazon Linux

macOS

Ubuntu

Windows

Red Hat

SUSE Linux

Debian

[Browse more AMIs](#)
Including AMIs from AWS, Marketplace and the Community

Summary

Number of instances [Info](#)

1

Software Image (AMI)
Canonical, Ubuntu, 26.04, amd64 resolute image
ami-0b6d9d3d33ba97d99

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

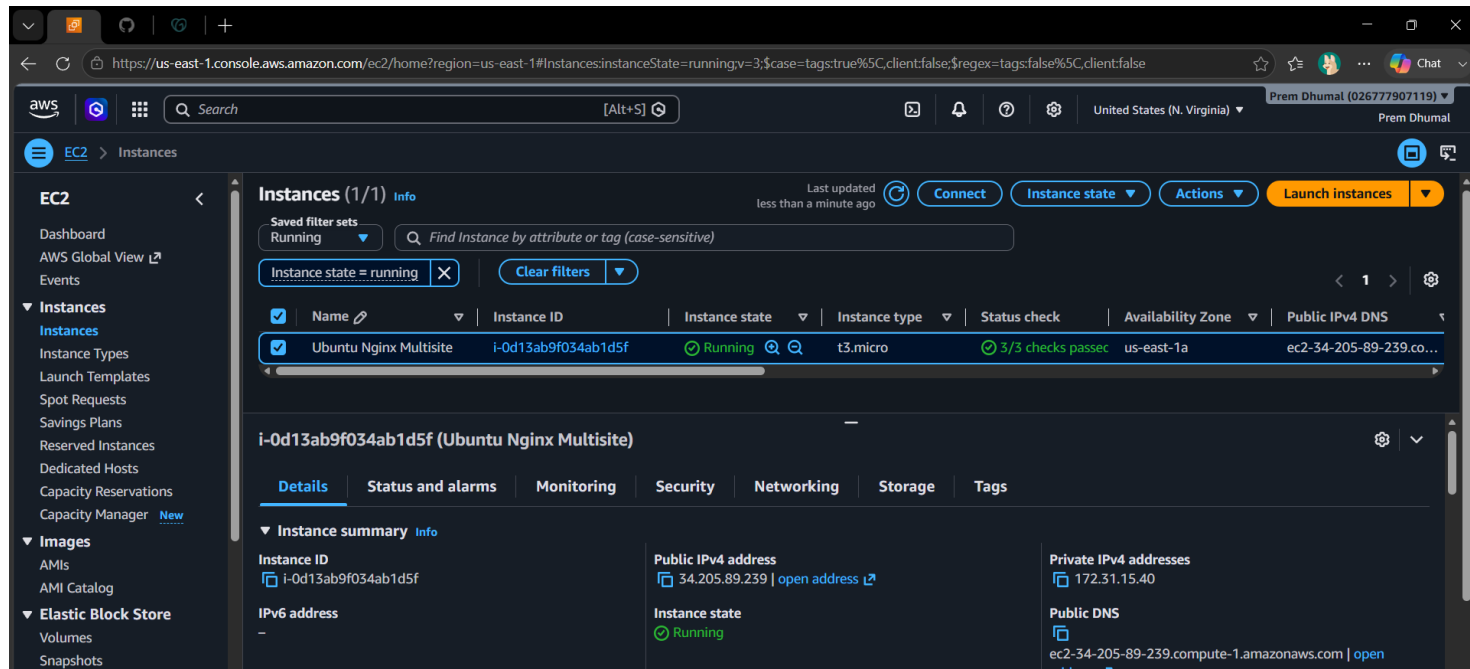
Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

Amazon Machine Image (AMI)

Ubuntu Server 26.04 LTS (HVM), SSD Volume Type Free tier eligible

Step 2 — Verify the EC2 Instance is Running



The screenshot displays the AWS Management Console interface for the EC2 service. The top navigation bar shows the AWS logo, a search bar, and the current region (United States (N. Virginia)). The left sidebar contains the navigation menu with categories like EC2, Images, and Elastic Block Store. The main content area is titled 'Instances (1/1)' and shows a single instance, 'Ubuntu Nginx Multisite', with the ID 'i-0d13ab9f034ab1d5f'. The instance is in the 'Running' state, as indicated by the green checkmark and the 'Running' label. The instance type is 't3.micro' and it is located in the 'us-east-1a' Availability Zone. The public IPv4 address is 'ec2-34-205-89-239.co...'. Below the instance list, the 'Details' tab is selected, showing the 'Instance summary' with fields for Instance ID, Public IPv4 address, Private IPv4 addresses, IPv6 address, Instance state, and Public DNS.

Instances (1/1) Info Last updated less than a minute ago [Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

Saved filter sets: Running

[Instance state = running](#) [Clear filters](#)

☒	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
☒	Ubuntu Nginx Multisite	i-0d13ab9f034ab1d5f	Running	t3.micro	3/3 checks passed	us-east-1a	ec2-34-205-89-239.co...

i-0d13ab9f034ab1d5f (Ubuntu Nginx Multisite)

[Details](#) [Status and alarms](#) [Monitoring](#) [Security](#) [Networking](#) [Storage](#) [Tags](#)

Instance summary Info

Instance ID i-0d13ab9f034ab1d5f	Public IPv4 address 34.205.89.239 open address	Private IPv4 addresses 172.31.15.40
IPv6 address -	Instance state Running	Public DNS ec2-34-205-89-239.compute-1.amazonaws.com open address

Step 3 — Connect to Ubuntu Using SSH

```
PS C:\Users\praje> ssh -i .\Downloads\ubuntu-multisite.pem ubuntu@34.205.89.239
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Aug 21 05:28:40 UTC 2026

System load:  0.04           Temperature:   -273.1 C
Usage of /:   30.6% of 6.61GB Processes:      121
Memory usage: 27%           Users logged in: 0
Swap usage:   0%            IPv4 address for ens5: 172.31.15.40

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Fri Aug 21 05:28:42 2026 from 152.58.33.195
ubuntu@ip-172-31-15-40:~$ pwd
/home/ubuntu
ubuntu@ip-172-31-15-40:~$ whoami
ubuntu
```

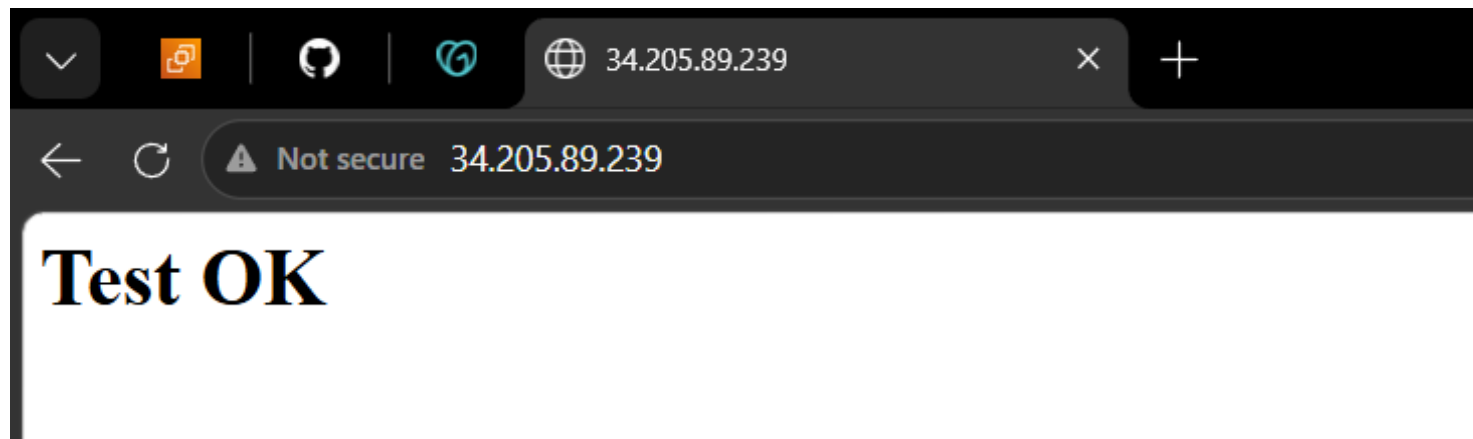
Step 4 — Create and Run the Shell Script

```
ubuntu@ip-172-31-15-40: ~  
ubuntu@ip-172-31-15-40:~$ sudo nano setup.sh  
ubuntu@ip-172-31-15-40:~$ sudo cat setup.sh  
sudo apt update -y  
sudo apt install nginx -y  
sudo systemctl start nginx  
sudo systemctl enable nginx  
echo "<h1>Test OK</h1>" | sudo tee /var/www/html/index.html  
sudo mkdir /var/www/html/travel  
sudo mkdir /var/www/html/hotel  
sudo mkdir /var/www/html/fitness  
sudo touch /var/www/html/travel/travel.html  
sudo touch /var/www/html/hotel/hotel.html  
sudo touch /var/www/html/fitness/fitness.html  
ubuntu@ip-172-31-15-40:~$ ls -l  
total 4  
-rw-r--r-- 1 root root 388 Aug 21 06:13 setup.sh  
ubuntu@ip-172-31-15-40:~$ sudo chmod +x setup.sh  
ubuntu@ip-172-31-15-40:~$ ls -l  
total 4  
-rwxr-xr-x 1 root root 388 Aug 21 06:13 setup.sh  
ubuntu@ip-172-31-15-40:~$ sudo ./setup.sh
```

Step 5 — Verify the Website Files

```
ubuntu@ip-172-31-15-40: /var  ×  +  ▾  
ubuntu@ip-172-31-15-40:~$ ls  
setup.sh  
ubuntu@ip-172-31-15-40:~$ cd /var/www/html/  
ubuntu@ip-172-31-15-40:/var/www/html$ ls  
fitness  hotel  index.html  travel  
ubuntu@ip-172-31-15-40:/var/www/html$ ls -R  
.:  
fitness  hotel  index.html  travel  
  
./fitness:  
fitness.html  
  
./hotel:  
hotel.html  
  
./travel:  
travel.html  
ubuntu@ip-172-31-15-40:/var/www/html$ sudo cat index.html  
<h1>Test OK</h1>  
ubuntu@ip-172-31-15-40:/var/www/html$ |
```

Step 6 — Confirm Shell Script Execution



Step 7 — Add the EC2 Server IP in GoDaddy DNS

The screenshot shows the GoDaddy DNS management interface in a web browser. The address bar displays `https://dcc.godaddy.com/control/dnsmanagement?domainName=premr.site`. On the left, a sidebar menu includes 'Portfolio', 'DNS' (highlighted), 'Transfers', 'Services', 'Investor Central', 'Settings', and 'Activity Log'. The main content area has a top section with three cards: 'ever with GoDaddy Airo.' (containing a 'Connect Domain' button), 'Verify Domain Ownership' (containing a 'Verify Domain Ownership' button), and 'your domain with email services.' (containing a 'Create Now' button). Below these is a section titled 'Add New Record' with a 'Filters' button and '7 records' listed. A table header shows 'Type', 'Name', 'Data', 'TTL', 'Propagation', 'Copy', 'Delete', and 'Edit'. A descriptive text block states: 'A records use an IP address to connect your domain to a website. They're also used to create subdomains such as www or store, that point to an IP address.' Below this is a form to add a new record with fields for 'Type' (set to 'A'), 'Name' (set to '@'), 'Value' (set to '34.205.89.239'), and 'TTL' (set to 'Custom' with a value of '600' seconds). 'Save' and 'Close' buttons are at the bottom right.

ever with GoDaddy Airo.

Connect Domain

Verify Domain Ownership

your domain with email services.

Create Now

Add New Record

Filters

7 records

Type	Name	Data	TTL	Propagation	Copy	Delete	Edit
------	------	------	-----	-------------	------	--------	------

A records use an IP address to connect your domain to a website. They're also used to create subdomains such as www or store, that point to an IP address.

Type *

A

Name *

@

Value *

34.205.89.239

TTL

Custom

Seconds

600

Save

Close

Step 8 — Create the Website Subdomains

Portfolio

DNS

Transfers

Services

Investor Central

Settings

Activity Log

New Records

CNAME records are a type of subdomain, or alias, that points to another domain name.

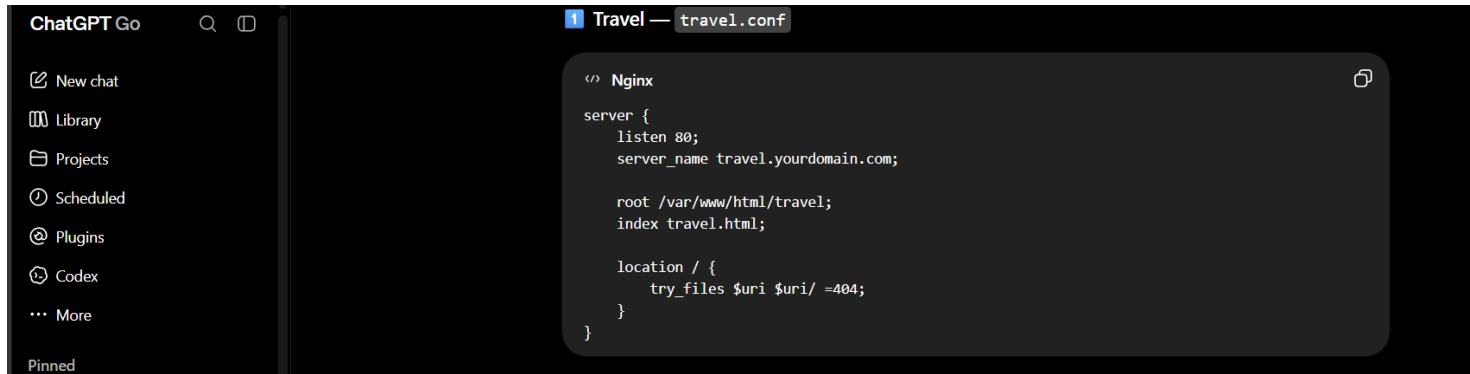
Type *	Name *	Value *	TTL
CNAME	travel	premrrr.site	1/2 Hour
CNAME	hotel	premrrr.site	1/2 Hour
CNAME	fitness	premrrr.site	1/2 Hour

Add More Records

Save All Records

Cancel

Step 9 — Prepare the Nginx Server Blocks



The screenshot shows a code editor interface with a dark theme. On the left is a sidebar with navigation options: 'ChatGPT Go' at the top, followed by 'New chat', 'Library', 'Projects', 'Scheduled', 'Plugins', 'Codex', and 'More'. At the bottom of the sidebar is a 'Pinned' section. The main editor area has a tab labeled '1 Travel — travel.conf'. Inside the editor, there is a code block titled 'Nginx' containing the following configuration:

```
</> Nginx

server {
    listen 80;
    server_name travel.yourdomain.com;

    root /var/www/html/travel;
    index travel.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Step 10 — Create and Configure the Nginx Files

```
ubuntu@ip-172-31-15-40: /etc × + ▾
ubuntu@ip-172-31-15-40:/etc/nginx/sites-enabled$ sudo nano {travel,hotel,fitness}.conf
ubuntu@ip-172-31-15-40:/etc/nginx/sites-enabled$ ls
default  fitness.conf  hotel.conf  travel.conf
ubuntu@ip-172-31-15-40:/etc/nginx/sites-enabled$ sudo cat travel.conf
server {
    listen 80;
    server_name travel.premrrr.site;

    root /var/www/html/travel/;
    index travel.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
ubuntu@ip-172-31-15-40:/etc/nginx/sites-enabled$ sudo cat hotel.conf
server {
    listen 80;
    server_name hotel.premrrr.site;

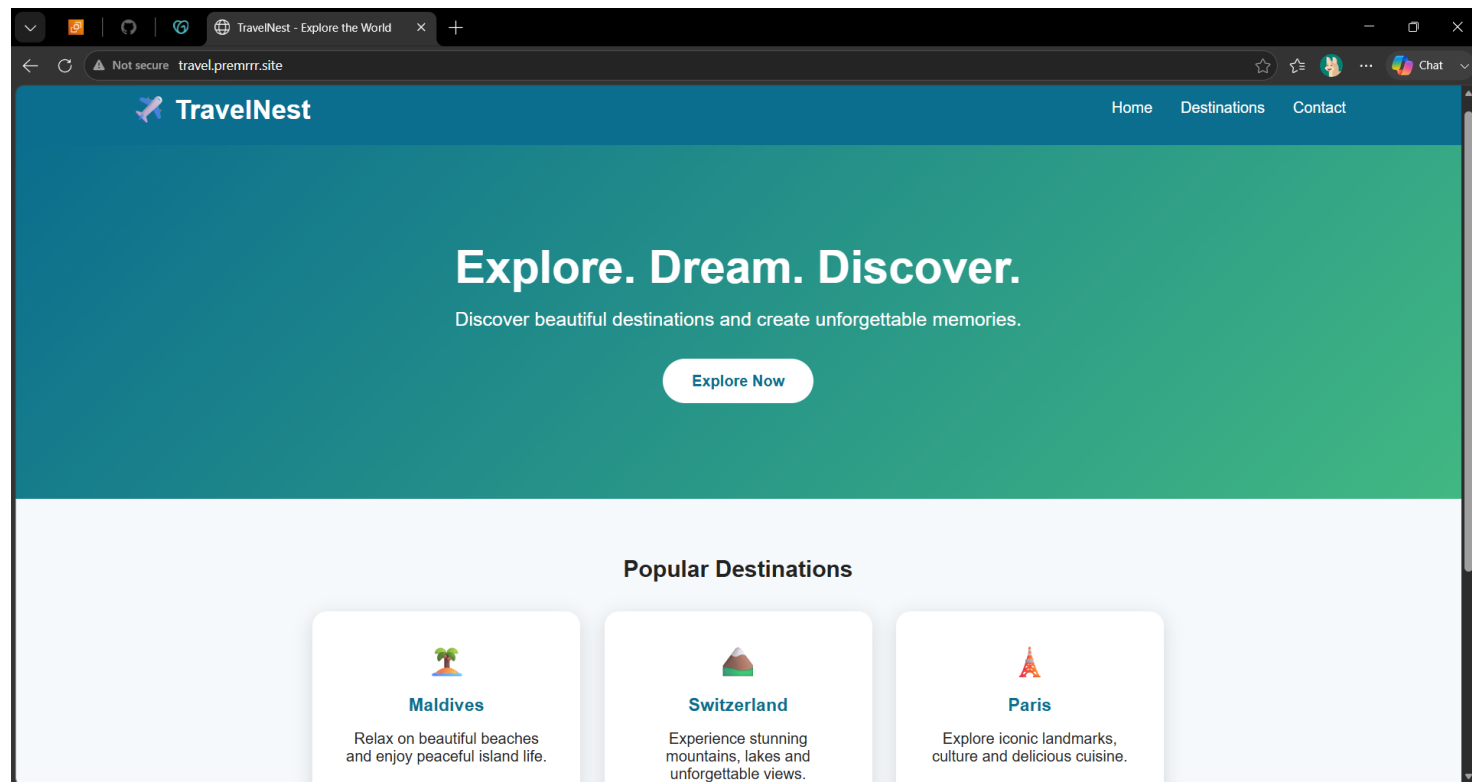
    root /var/www/html/hotel/;
    index hotel.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
ubuntu@ip-172-31-15-40:/etc/nginx/sites-enabled$ sudo cat fitness.conf
server {
    listen 80;
    server_name fitness.premrrr.site;

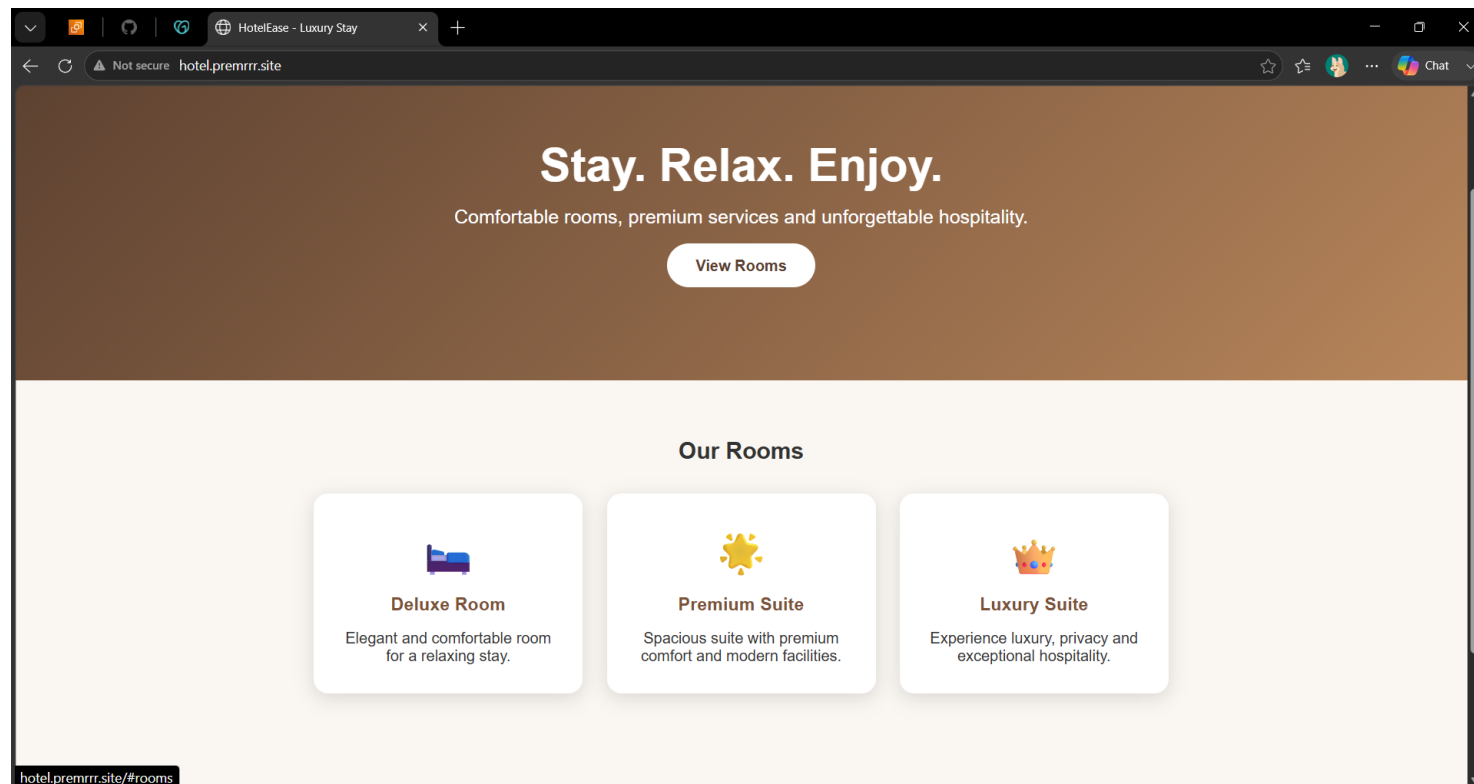
    root /var/www/html/fitness/;
    index fitness.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
ubuntu@ip-172-31-15-40:/etc/nginx/sites-enabled$ sudo service nginx reload
ubuntu@ip-172-31-15-40:/etc/nginx/sites-enabled$ |
```

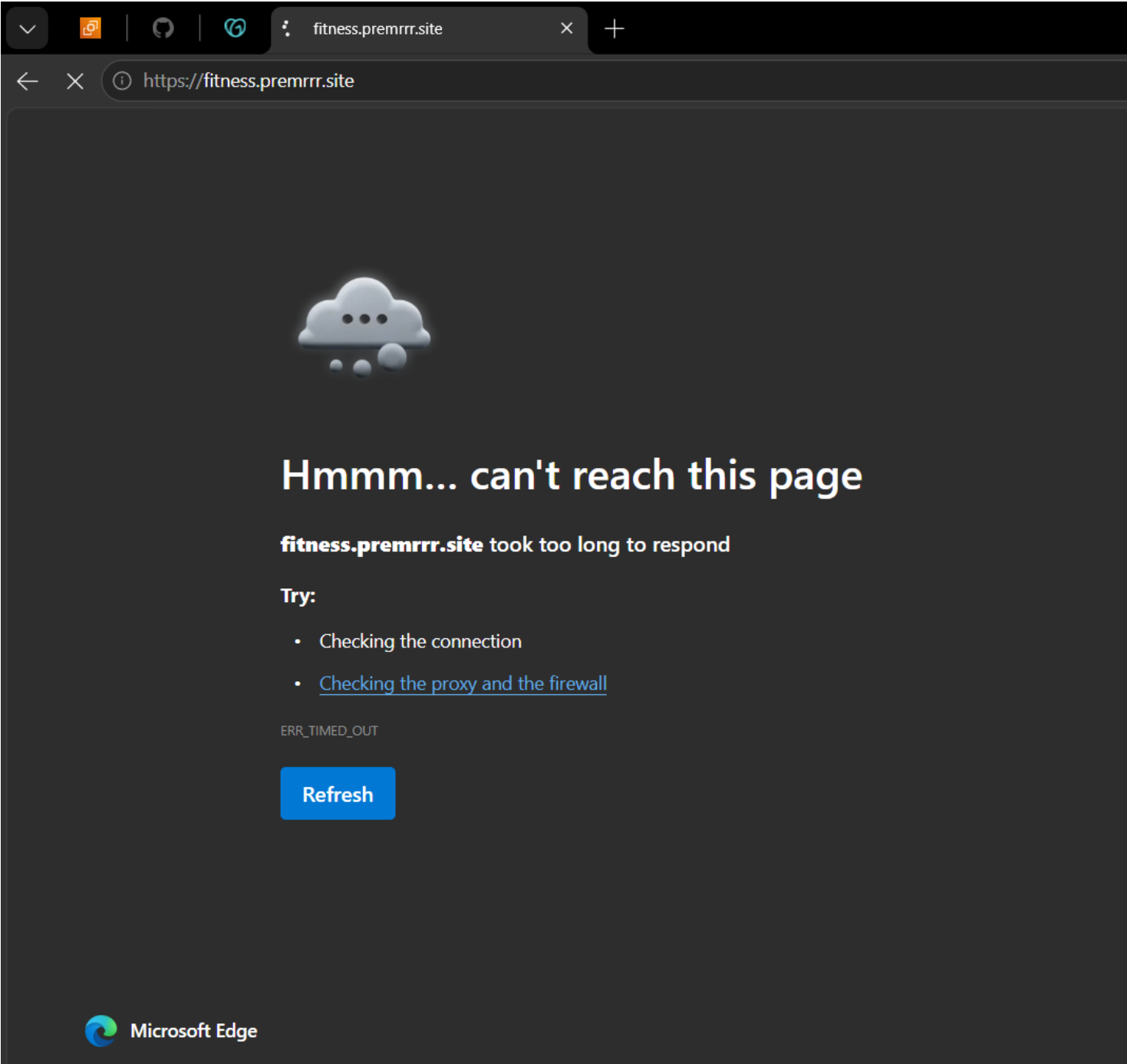
Step 11 — Verify the Travel Website



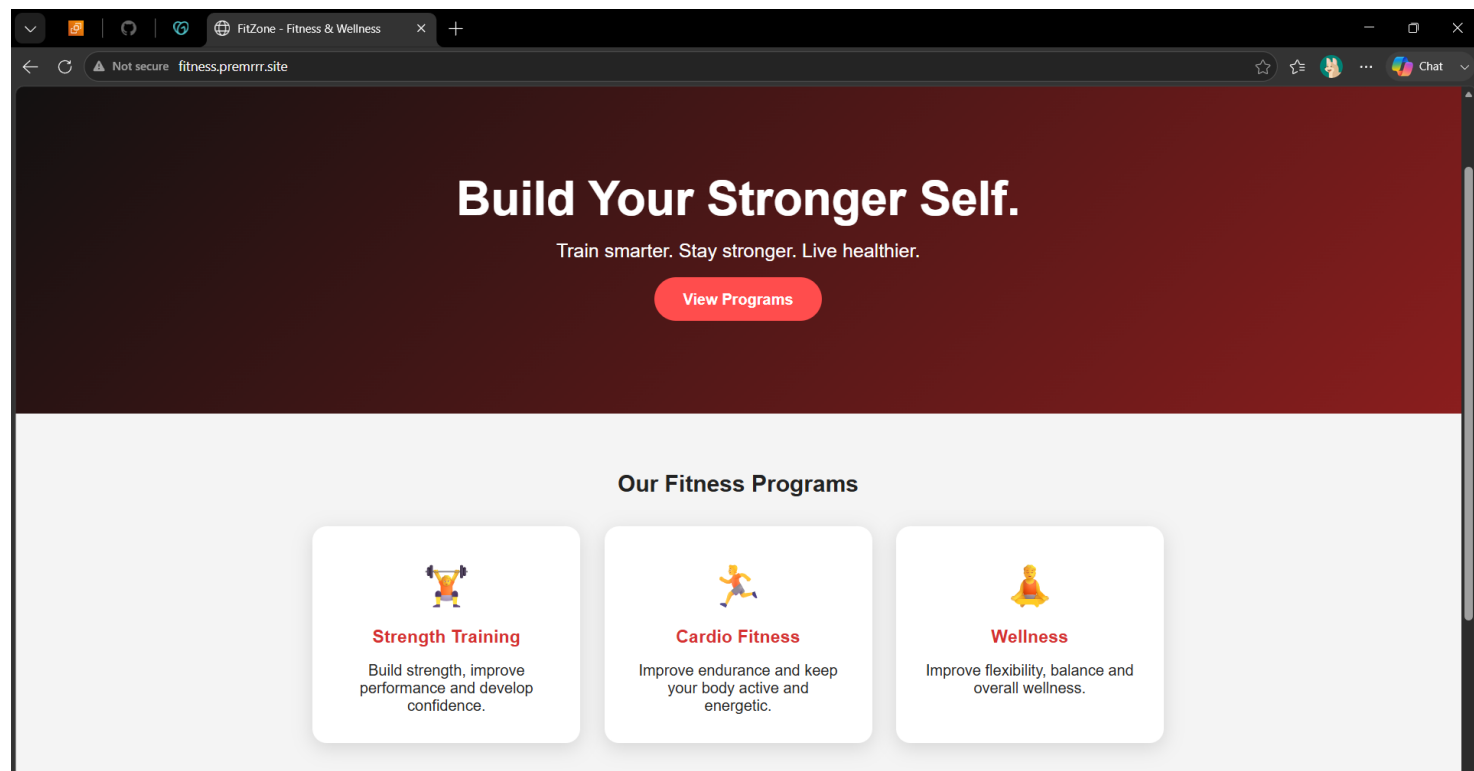
Step 12 — Verify the Hotel Website



Step 13 — Identify the HTTPS Access Issue



Step 14 — Verify the Fitness Website



Problems, Solutions & Conclusion

Problem 1 — Website access / 403

Observed: A 403 Forbidden response was encountered during website testing.

Solution: Check the Nginx server-block configuration, website root path and access permissions, then retest the configured site.

Problem 2 — HTTPS access

Observed: The project includes an HTTPS access test, but SSL/HTTPS is not configured in the deployment.

Solution: Use the configured HTTP subdomains for this project. HTTPS can be added later by configuring an SSL certificate and Nginx HTTPS settings.

Problem 3 — Multiple sites on one server

Observed: A single EC2 instance must serve different websites without mixing their content.

Solution: Use separate website directories and separate Nginx server blocks, with each hostname mapped to its own root.

Conclusion

This project demonstrates a practical multi-site deployment on Ubuntu using Nginx.

One EC2 server can host Travel, Hotel and Fitness websites when DNS and server blocks are configured correctly.

The project builds hands-on understanding of EC2, Ubuntu, Nginx, DNS, subdomains, shell scripting and basic web troubleshooting.